

甚至不知道所处理的对象的类型。然而一旦跨入 STL 你就会了解, 迭代器和函数对象都是在 C 指针之上塑造出来的, 所以对 STL 的迭代器和函数对象而言, 旧式的 C *pass-by-value* 守则再次适用 (参数传递方式的选择细节请见条款 20)。

因此我说, C++ 并不是一个带有一组守则的一体语言; 它是从四个次语言组成的联邦政府, 每个次语言都有自己的规约。记住这四个次语言你就会发现 C++ 容易了解得多。

请记住

- C++ 高效编程守则视状况而变化, 取决于你使用 C++ 的哪一部分。

条款 02: 尽量以 const, enum, inline 替换 #define

Prefer consts, enums, and inlines to #defines.

这个条款或许改为“宁可以编译器替换预处理器”比较好, 因为或许 #define 不被视为语言的一部分。那正是它的问题所在。当你做出这样的事情:

```
#define ASPECT_RATIO 1.653
```

记号名称 ASPECT_RATIO 也许从未被编译器看见; 也许在编译器开始处理源码之前它就被预处理器移走了。于是记号名称 ASPECT_RATIO 有可能没进入记号表 (symbol table) 内。于是当你运用此常量但获得一个编译错误信息时, 可能会带来困惑, 因为这个错误信息也许会提到 1.653 而不是 ASPECT_RATIO。如果 ASPECT_RATIO 被定义在一个非你所写的头文件内, 你肯定对 1.653 以及它来自何处毫无概念, 于是你将因为追踪它而浪费时间。这个问题也可能出现在记号式调试器 (symbolic debugger) 中, 原因相同: 你所使用的名称可能并未进入记号表 (symbol table)。

解决之道是以一个常量替换上述的宏 (#define):

```
const double AspectRatio = 1.653;    //大写名称通常用于宏,  
                                     //因此这里改变名称写法。
```

作为一个语言常量, AspectRatio 肯定会被编译器看到, 当然就会进入记号表内。此外对浮点常量 (floating point constant, 就像本例) 而言, 使用常量可能比使用 #define 导致较小量的码, 因为预处理器“盲目地将宏名称 ASPECT_RATIO 替换为 1.653”可能导致目标码 (object code) 出现多份 1.653, 若改用常量 AspectRatio 绝不会出现相同情况。

当我们以常量替换 `#defines`，有两种特殊情况值得说说。第一是定义常量指针（`constant pointers`）。由于常量定义式通常被放在头文件内（以便被不同的源码含入），因此有必要将指针（而不只是指针所指之物）声明为 `const`。例如若要在头文件内定义一个常量的（不变的）`char*-based` 字符串，你必须写 `const` 两次：

```
const char* const authorName = "Scott Meyers";
```

关于 `const` 的意义和使用（特别是当它与指针结合时），条款 3 有完整的讨论。这里值得先提醒你的是，`string` 对象通常比其前辈 `char*-based` 合宜，所以上述的 `authorName` 往往定义成这样更好些：

```
const std::string authorName("Scott Meyers");
```

第二个值得注意的是 `class` 专属常量。为了将常量的作用域（`scope`）限制于 `class` 内，你必须让它成为 `class` 的一个成员（`member`）；而为确保此常量至多只有一份实体，你必须让它成为一个 `static` 成员：

```
class GamePlayer {
private:
    static const int NumTurns = 5;    //常量声明式
    int scores[NumTurns];            //使用该常量
    ...
};
```

然而你所看到的是 `NumTurns` 的声明式而非定义式。通常 C++ 要求你对你所使用的任何东西提供一个定义式，但如果它是个 `class` 专属常量又是 `static` 且为整数类型（`integral type`，例如 `ints`, `chars`, `bools`），则需特殊处理。只要不取它们的地址，你可以声明并使用它们而无须提供定义式。但如果你取某个 `class` 专属常量的地址，或纵使你不取其地址而你的编译器却（不正确地）坚持要看到一个定义式，你就必须另外提供定义式如下：

```
const int GamePlayer::NumTurns; //NumTurns 的定义;
                                //下面告诉你为什么没有给予数值
```

请把这个式子放进一个实现文件而非头文件。由于 `class` 常量已在声明时获得初值（例如先前声明 `NumTurns` 时为它设初值 5），因此定义时不可以再设初值。

顺带一提，请注意，我们无法利用 `#define` 创建一个 `class` 专属常量，因为 `#defines` 并不重视作用域（`scope`）。一旦宏被定义，它就在其后的编译过程中有

效（除非在某处被`#undef`）。这意味`#defines`不仅不能够用来定义 `class` 专属常量，也不能够提供任何封装性，也就是说没有所谓 `private #define` 这样的东西。而当然 `const` 成员变量是可以被封装的，`NumTurns` 就是。

旧式编译器也许不支持上述语法，它们不允许 `static` 成员在其声明式上获得初值。此外所谓的“`in-class` 初值设定”也只允许对整数常量进行。如果你的编译器不支持上述语法，你可以将初值放在定义式：

```
class CostEstimate {
private:
    static const double FudgeFactor;    //static class 常量声明
    ...                                //位于头文件内
};
const double                                //static class 常量定义
    CostEstimate::FudgeFactor = 1.35;    //位于实现文件内
```

这几乎是你在任何时候唯一需要做的事。唯一例外是当你在 `class` 编译期间需要一个 `class` 常量值，例如在上述的 `GamePlayer::scores` 的数组声明式中（是的，编译器坚持必须在编译期间知道数组的大小）。这时候万一你的编译器（错误地）不允许“`static` 整数型 `class` 常量”完成“`in class` 初值设定”，可改用所谓的“`the enum hack`”补偿做法。其理论基础是：“一个属于枚举类型（`enumerated type`）的数值可权充 `ints` 被使用”，于是 `GamePlayer` 可定义如下：

```
class GamePlayer {
private:
    enum { NumTurns = 5 };                //“the enum hack”—— 令 NumTurns
    ...                                //成为 5 的一个记号名称.
    int scores[NumTurns];                //这就没问题了.
    ...
};
```

基于数个理由 `enum hack` 值得我们认识。第一，`enum hack` 的行为某方面说比较像 `#define` 而不像 `const`，有时候这正是你想要的。例如取一个 `const` 的地址是合法的，但取一个 `enum` 的地址就不合法，而取一个 `#define` 的地址通常也不合法。

如果你不想让别人获得一个 `pointer` 或 `reference` 指向你的某个整数常量, `enum` 可以帮助你实现这个约束。(条款 18 对于“通过撰码时的决定实施设计上的约束条件”谈得更多。)此外虽然优秀的编译器不会为“整数型 `const` 对象”设定另外的存储空间(除非你创建一个 `pointer` 或 `reference` 指向该对象),不够优秀的编译器却可能如此,而这可能是你不想要的。`Enums` 和 `#defines` 一样绝不会导致非必要的内存分配。

认识 `enum hack` 的第二个理由纯粹是为了实用主义。许多代码用了它,所以看到它时你必须认识它。事实上 “`enum hack`” 是 `template metaprogramming` (模板元编程, 见条款 48) 的基础技术。

把焦点拉回预处理器。另一个常见的 `#define` 误用情况是以它实现宏(`macros`)。宏看起来像函数,但不会招致函数调用(`function call`)带来的额外开销。下面这个宏夹带着宏实参,调用函数 `f`:

```
//以 a 和 b 的较大值调用 f
#define CALL_WITH_MAX(a, b) f((a) > (b) ? (a) : (b))
```

这般长相的宏有着太多缺点,光是想到它们就让人痛苦不堪。

无论何时当你写出这种宏,你必须记住为宏中的所有实参加上小括号,否则某些人在表达式中调用这个宏时可能会遭遇麻烦。但纵使你为所有实参加上小括号,看看下面不可思议的事情:

```
int a = 5, b = 0;
CALL_WITH_MAX(++a, b);           //a 被累加二次
CALL_WITH_MAX(++a, b+10);        //a 被累加一次
```

在这里,调用 `f` 之前, `a` 的递增次数竟然取决于“它被拿来和谁比较”!

幸运的是你不需要对这种无聊事情提供温床。你可以获得宏带来的效率以及一般函数的所有可预料行为和类型安全性(`type safety`)——只要你写出 `template inline` 函数(见条款 30):

```
template<typename T>                               //由于我们不知道
inline void callWithMax(const T& a, const T& b) //T 是什么,所以采用
{                                                  //pass by reference-to-const.
    f(a > b ? a : b);                             //见条款 20.
}
```

这个 `template` 产出一整群函数,每个函数都接受两个同型对象,并以其中较大

者调用 `f`。这里不需要在函数本体中为参数加上括号,也不需要操心参数被核算(求值)多次……等等。此外由于 `callWithMax` 是个真正的函数,它遵守作用域(scope)和访问规则。例如你绝对可以写出一个“class 内的 `private inline` 函数”。一般而言宏无法完成此事。

有了 `consts`、`enums` 和 `inlines`,我们对预处理器(特别是`#define`)的需求降低了,但并非完全消除。`#include` 仍然是必需品,而`#ifdef/#ifndef`也继续扮演控制编译的重要角色。目前还不到预处理器全面引退的时候,但你应该明确地给予它更长更频繁的假期。

请记住

- 对于单纯常量,最好以 `const` 对象或 `enums` 替换`#defines`。
- 对于形似函数的宏(macros),最好改用 `inline` 函数替换`#defines`。

条款 03: 尽可能使用 const

Use const whenever possible.

`const` 的一件奇妙事情是,它允许你指定一个语义约束(也就是指定一个“不该被改动”的对象),而编译器会强制实施这项约束。它允许你告诉编译器和其他程序员某值应该保持不变。只要这(某值保持不变)是事实,你就该确实说出来,因为说出来可以获得编译器的襄助,确保这条约束不被违反。

关键字 `const` 多才多艺。你可以用它在 `classes` 外部修饰 `global` 或 `namespace`(见条款 2)作用域中的常量,或修饰文件、函数、或区块作用域(block scope)中被声明为 `static` 的对象。你也可以用它修饰 `classes` 内部的 `static` 和 `non-static` 成员变量。面对指针,你也可以指出指针自身、指针所指物,或两者都(或都不)是 `const`:

```
char greeting[] = "Hello";  
char* p = greeting;           //non-const pointer, non-const data  
const char* p = greeting;     //non-const pointer, const data  
char* const p = greeting;     //const pointer, non-const data  
const char* const p = greeting; //const pointer, const data
```